Member 5 — Web Interface Integration Handoff

Project: Automated Municipal Ticket System
Owner of n8n orchestration: Member 1
Recipient: Member 5 — Citizen web app, municipality dashboard and database
Baseline: Working Phase 5, three-agent **demo** workflow, as tested by Member 1
Contract status: Proposed; all team members should confirm before freezing.

> **Read this first:** The current n8n workflow accepts JSON reports referring to **mock image fixtures**, calls three separate **simulated** HTTP agent services, applies validation gates and deterministic routing, and returns PASS / REVIEW / REJECT. It does **not** upload or analyze real photographs, retrieve verified municipal policies, or save tickets in a database. A PASS contains `ticket.persisted=false`.

1. What Member 1 built

- Windows 11 + Docker Desktop + self-hosted n8n, using the `municipal-net` Docker network.
- An importable **20-node Phase 5 workflow**: `n8n/municipal-ticket-phase5.json`.
- Input checks; Intake, Vision and Knowledge HTTP integrations; three policy gates; department routing; demo ticket formatting; trace/audit summary; single webhook response.
- Agent HTTP nodes retry up to 3 **total attempts** with 1-second spacing; service failures/malformed outputs go to REVIEW, not silent PASS.
- Verified demo PASS / REVIEW / REJECT, Intake failure tests, Vision outage and recovery, and a working Phase 5 PASS response.

2. Whole system architecture (current vs target)

```
CURRENT DEMO
Citizen UI (Member 5: not yet connected)
       |
       | JSON report (report_id, description, media_ref, location?)
       v
n8n Phase 5 Webhook (Member 1)
       |
       +--> Input Check --> REJECT on invalid fields
       |
       +--> HTTP Intake stub (Member 3 interface) --> Gate 1 --> REVIEW if invalid
       |
       +--> HTTP Vision stub (Member 4 interface) --> Gate 2 --> REVIEW if uncertain
       |
       +--> Fixed department routing (Member 1; not an AI decision)
       |
       +--> HTTP Knowledge stub (Member 2 interface) --> Gate 3 --> REVIEW on poor sources
       |
       +--> Mock Ticket Creation --> Audit Summary --> JSON response
                                                  |
                                                  v
                                    Citizen UI displays result

TARGET TEAM SYSTEM (NOT COMPLETE)
Citizen UI --> Member 5 authenticated backend / upload storage --> n8n
    Intake (actual image checks + anonymization) --> Vision (real model)
    --> deterministic routing --> Knowledge (RAG + verified sources)
    --> Member 5 ticket-storage API / Postgres + event log
    --> staff dashboard + human review / ticket updates
```

Responsibility boundary: n8n orchestrates and decides PASS/REVIEW/REJECT, and selects the department using rules. Member 5 builds forms, backend/proxy, uploads, Postgres persistence, review queue, dashboard and ticket-status updates. Members 2/3/4 own their AI/processing services.

3. Exact Phase 5 endpoints

Audience	Method and URL	Status
Citizen app → n8n, test	`POST http://localhost:5678/webhook-test/municipal-ticket-phase5`	Works **only while "Listen for Test Event" is active**
Citizen app → n8n, published	`POST http://localhost:5678/webhook/municipal-ticket-phase5`	Works after Phase 5 workflow is **activated/published**
n8n → Intake	`POST http://municipal-intake-stub:8001/intake`	Running stub, Docker-internal
n8n → Vision	`POST http://municipal-vision-stub:8002/vision`	Running stub, Docker-internal
n8n → Knowledge	`POST http://municipal-knowledge-stub:8003/knowledge`	Running stub, Docker-internal
Developer health checks	`GET http://localhost:8001/health`, `:8002/health`, `:8003/health`	On Member 1's PC only
Member 5 upload & ticket endpoints	`POST /uploads`, `POST /tickets`, `GET /tickets`, `PATCH /tickets/{id}/status`	**Proposed, not implemented**

IMPORTANT — outdated docs path: The original `docs/contracts.md` contains `POST /webhook/municipal-ticket` as an earlier general example. The **actual exported Phase 5 workflow** uses the `municipal-ticket-phase5` path above. Use the actual path for the demo. `localhost` means **the machine making the request**: on Member 5's laptop it will not point to Member 1's n8n. Share a reachable, secured endpoint or arrange a backend-to-backend connection before remote testing.

4. Citizen request contract (current)

Method: `POST`. Header: `Content-Type: application/json`. **No multipart image upload is implemented in Phase 5.**

```
{
  "report_id": "RPT-001",
  "description": "A deep pothole is dangerous for drivers on the road.",
  "media_ref": "mock://pothole",
  "location": {
    "address": "Demo Avenue (fictional)",
    "latitude": null,
    "longitude": null
  }
}
```

Field	Type	Required	Validations in Phase 5
`report_id`	string	yes	3–64 characters; letters/digits/_/-; frontend or backend generates a unique ID
`description`	string	yes	5–3000 trimmed characters
`media_ref`	string	yes	`mock://...` demo fixture or `uploads/<safe-path>`; no arbitrary URLs, no `..`, <=512 chars
`location`	object or null	no	`address` max 300 characters; `latitude` range -90...90 or null; `longitude` -180...180 or null

Demo-only fixtures: `mock://pothole`, `mock://streetlight`, `mock://waste`, `mock://water_leak`, `mock://unrelated`, `mock://blurry`. These are **labels, not image files**. Sending an `uploads/...` reference with stub Intake does **not** validate an image; the workflow safely returns REVIEW.

5. Citizen response contract

The webhook returns **one JSON object**, not the n8n canvas array wrapper. The response always contains `report_id`, `decision`, `ticket`, `review_reasons`, `errors`, `demo_mode`, `agents_are_mock`, `notice`, `versions`, `trace`, `audit`, and `http_status`.

PASS — HTTP 200

Selected fields from a valid pothole fixture:

```
{
  "report_id": "RPT-001",
  "decision": "PASS",
  "ticket": {
    "ticket_id": "DEMO-TKT-RPT-001",
    "report_id": "RPT-001",
    "status": "TICKET_CREATED",
    "category": "ROAD_DAMAGE",
    "severity": "HIGH",
    "priority": "HIGH",
    "department": "ROADS_DEPARTMENT",
    "anonymized_media_ref": null,
    "deadline": null,
    "persisted": false,
    "storage_status": "MOCK_ONLY",
    "is_mock": true
  },
  "review_reasons": [],
  "errors": [],
  "demo_mode": true,
  "agents_are_mock": true,
  "http_status": 200
}
```


The full response also has `ticket.location`, `ticket.procedure`, `ticket.sources`, `notice`, `versions`, `trace` and `audit`. The UI must display **“Demo result — not saved”** rather than a real tracking confirmation when `persisted:false`.

REVIEW — HTTP 202


```
{
  "report_id": "RPT-002",
  "decision": "REVIEW",
  "ticket": null,
  "review_reasons": ["IMAGE_TEXT_MISMATCH_OR_UNCLEAR"],
  "errors": [],
  "demo_mode": true,
  "http_status": 202
}
```


This is a **selected-fields example**, not the complete returned object. Show a “Needs human review” state and the reason. **No review record is currently persisted.** A future backend must save REVIEW cases for the staff dashboard.

REJECT — HTTP 422


```
{
  "report_id": "RPT-003",
  "decision": "REJECT",
  "ticket": null,
  "review_reasons": [],
  "errors": ["MISSING_MEDIA_REF"],
  "demo_mode": true,
  "http_status": 422
}
```


Show a corrective message (e.g., “Please select a demo image fixture”). HTTP 202 and 422 are **meaningful workflow outcomes**, not necessarily transport failures. Parse the JSON body even when `response.ok` is false.

6. Domain values for UI and database

Kind	Allowed values
`decision`	`PASS`, `REVIEW`, `REJECT`
`category`	`ROAD_DAMAGE`, `STREET_LIGHTING`, `WASTE`, `WATER_LEAK`, `NONE`
`severity`	`LOW`, `MEDIUM`, `HIGH`
`priority`	`LOW`, `NORMAL`, `HIGH`
`department`	`ROADS_DEPARTMENT`, `PUBLIC_LIGHTING_DEPARTMENT`, `SANITATION_DEPARTMENT`, `WATER_SERVICES_DEPARTMENT`
Target ticket `status`	`RECEIVED`, `VALIDATING`, `ANALYZING`, `REVIEW_REQUIRED`, `TICKET_CREATED`, `ASSIGNED`, `IN_PROGRESS`, `RESOLVED`, `REJECTED`

Routing is **rule-based**, not decided by an LLM:

Vision category	Department
`ROAD_DAMAGE`	`ROADS_DEPARTMENT`
`STREET_LIGHTING`	`PUBLIC_LIGHTING_DEPARTMENT`
`WASTE`	`SANITATION_DEPARTMENT`
`WATER_LEAK`	`WATER_SERVICES_DEPARTMENT`
`NONE` / unknown	REVIEW, do not route

These are **internal demo enums**, not official department names for a verified municipality. In the frontend, use clear human-friendly labels but preserve the enum values in API calls/database.

7. n8n architecture — 20 nodes

```
Citizen Webhook
  -> Input Check -> Input PASS?
     False -> Audit Summary -> Format API Response -> Respond to Citizen
     True  -> HTTP Intake -> Attach Intake Response -> Gate 1 - Intake -> Intake PASS?
              False -> Audit Summary -> Format API Response -> Respond to Citizen
              True  -> HTTP Vision -> Attach Vision Response -> Gate 2 - Vision -> Vision PASS?
                       False -> Audit Summary -> Format API Response -> Respond to Citizen
                       True  -> Department Rules -> HTTP Knowledge -> Attach Knowledge Response
                                -> Gate 3 - Sources -> Sources PASS?
                                   False -> Audit Summary -> Format API Response -> Respond to Citizen
                                   True  -> Mock Ticket Creation -> Audit Summary
                                            -> Format API Response -> Respond to Citizen
```

Stage	Job	Key rejection/review behavior
Input Check	Validate fields and reject unsafe media references	REJECT: invalid ID/description/missing or unsafe media
HTTP Intake + Gate 1	Normalize and check media/privacy status	REVIEW: unreachable/invalid response, wrong ID, unverified real image
HTTP Vision + Gate 2	Classify and check evidence/mismatch	REVIEW: unknown category, missing evidence, uncertain or mismatch

Stage	Job	Key rejection/review behavior
Department Rules	Assign internal department by category	REVIEW if no mapping
HTTP Knowledge + Gate 3	Obtain demo procedure/sources and validate provenance	REVIEW: missing or mismatched sources, unsupported official claims
Mock Ticket Creation	Create JSON representation **only**	`persisted:false` and `storage_status:MOCK_ONLY`
Audit Summary	Build in-memory execution trace	**Not permanent database logging**
Respond to Citizen	Return final JSON with HTTP 200/202/422	One endpoint for all decisions

n8n resilience: HTTP Intake timeout 5s, Vision 10s, Knowledge 8s; each HTTP node is configured for 3 total tries with 1s between tries. `onError: continueRegularOutput` allows the gate to turn service errors into REVIEW. Some timeouts can cause multiple seconds' delay in the UI; show a loading state.

8. Member 5 — what the web interface should implement

Citizen page — current demo: Report description, optional location, **a clearly labeled demo fixture selector** instead of a real image uploader, Submit button, pending/loading UI, and three result panels: PASS demo ticket, REVIEW explanation, REJECT validation errors. Keep the `report_id` for correlation. Do not claim real image processing.

Dashboard — target: Tickets list with category, severity, priority, department, status, anonymized-media reference, source metadata and trace; filters; review queue; Approve/Reject/Assign/Resolve actions. No dashboard storage or status-update API exists yet; build mock screens or database endpoints independently until integrated.

Backend and storage (Member 5 to implement): PostgreSQL tables for `reports`, `tickets`, `ticket_events` (plus `review_cases` if appropriate); enforce unique `report_id` / idempotent saves; assign a real DB-generated `ticket_id` only after commit; protect raw images; store safe references, not image blobs; define who saves REVIEW cases; authenticate municipality users; limit CORS, uploads and request rates.

Proposed new backend endpoints — NOT CURRENTLY IMPLEMENTED:

Endpoint	Purpose	Required behavior
`POST /uploads`	Upload image bytes to controlled storage	Return safe media reference; Member 3 must later validate/ anonymize
`POST /tickets`	Save PASS ticket from n8n	Idempotent by `report_id`; return real `ticket_id` and `persisted:true` after commit
`POST /reviews`	Save REVIEW case	Persist reason codes and state for staff follow-up
`GET /tickets`	Dashboard list	Pagination/filters/authentication
`GET /tickets/{id}`	Ticket details	Include RAG provenance and events as available
`PATCH /tickets/{id}/ status`	Staff actions	Validate allowed transitions and append `ticket_events`

Integration decision to agree together: Prefer Member 5 backend as the only database writer; Member 1 can add a POST to Member 5's authenticated ticket endpoint immediately after Gate 3, replacing `Mock Ticket Creation`, with DB failures producing a safe non-success result. Avoid both n8n and the frontend separately writing the same ticket (duplicate risk). Reviews also need a durable queue.

9. Frontend integration in Windows development

Easy local demo: If Member 5 runs the frontend on **the same PC as n8n**, call the test webhook while `Listen for Test Event` is active. For repeated requests, activate the Phase 5 workflow and use the production webhook path. A browser may be blocked by CORS. Recommended: **React/Vite calls its own backend or Vite development proxy**, and that server talks to n8n. Never embed n8n admin credentials or private API keys in browser JavaScript.

Example curl (run from extracted project folder):

```
curl.exe -i -X POST "http://localhost:5678/webhook-test/municipal-ticket-phase5" -H "Content-Type: application/json" --da
ta-binary "@examples/pass_pothole.json"
```

React/TypeScript calling a backend proxy (see `frontend/n8n-client.ts`):

```
const response = await fetch('/api/municipal-ticket', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    report_id: 'RPT-UI-001',
    description: 'Deep pothole on roadway',
    media_ref: 'mock://pothole',
    location: null
  })
});
const result = await response.json();
// Handle result.decision even if response.status is 202 or 422.
```

If Member 5 runs on another machine, do **not** use their `localhost:5678`. Arrange a secure reachable service/remote backend (or deploy both app and n8n to the same host/network). Do not open an unauthenticated webhook to the public Internet for the demo. `8001/8002/8003` are agent ports for Member 1's local diagnostic use; the frontend should normally call only its own API, never internal agent URLs.

10. Tests Member 5 can reproduce

Before **each** request to the test webhook, click “Listen for Test Event” in n8n.

Fixture	Expected HTTP	Decision	Ticket
`examples/pass_pothole.json`	200	PASS	Simulated ticket; `persisted:false`
`examples/review_mismatch.json`	202	REVIEW	null
`examples/reject_missing_media.json`	422	REJECT	null
`examples/knowledge_outage.json`	202	REVIEW	null
`examples/knowledge_empty_sources.json`	202	REVIEW	null

Already verified by Member 1: PASS/REVIEW/REJECT and Intake failure tests, Phase 4 Vision outage response and recovery; Member 1 reported a working Phase 5 PASS with all three HTTP agents. Other Phase 5 negative cases are provided as fixtures and should be run on the integrated frontend; they have not all been confirmed live in this conversation.

11. Questions to settle before Member 5 wires up the UI

1. Is the first UI demo on the same Windows computer as Member 1's n8n, or does it need a secure remote endpoint?

2. Will Member 5 provide a backend/proxy endpoint `/api/municipal-ticket` that relays the JSON to n8n?
3. Who creates the unique `report_id` and where do we prevent duplicate submissions?
4. Should REVIEW submissions be saved immediately to a `review_cases` table even if no ticket exists?
5. Which upload-storage API and safe-image reference format will Member 3 consume?
6. What are the exact real ticket save/read/status-update API contracts and authentication method?
7. When will Members 2–4 replace demo services and confirm the shared JSON field names?

12. Package contents / source of truth

- `n8n/municipal-ticket-phase5.json` — **actual importable working demo workflow**, not just a diagram.
- `docs/contracts-original.md` — original proposed team contract. **Webhook path is earlier example; Phase 5 path in this handoff is authoritative for the demo.**
- `docs/architecture-original.md` — architecture research by Member 1.
- `examples/*.json` — runnable request fixtures.
- `frontend/n8n-client.ts` — frontend-facing request/response types and fetch helper (assumes a server-side proxy at `/api/municipal-ticket`).
- `README_START_HERE.md` — this document.

Do not treat demo output as municipal policy, verified image evidence or persisted tickets. Confirm the API contract with the team before freezing the public interface.

Historical source packages (included)

- `source_packages/Phase5_full_three_agent_demo.zip` — complete original Phase 5 source (workflow, contracts, Knowledge stub, tests and instructions).
- `source_packages/Phase4_intake_vision_demo.zip` — Vision stub source and Phase 4 workflow/tests.
- `source_packages/Phase2_intake_http_demo.zip` — Intake stub source and Phase 2 workflow/tests.

These archive snapshots are included for code inspection and reproducibility. **Use the Phase 5 workflow** for new frontend integration; the older stages remain references only.